

Backend/Server Setup (Server Configuration)

Story: Backend Setup (Server Configuration)

Estimated Duration: 15 Minutes

Description

The **Backend** of the **House Rent Management System** is developed using **Node.js** and **Express.js**. The backend is responsible for handling client requests, connecting to the MongoDB database, managing APIs, and implementing the business logic of the application. Follow the steps below to set up and run the backend server.

Step 1: Open the Server Folder

Open **Visual Studio Code** and launch the **Integrated Terminal**.

Navigate to the **server** folder where the backend application will be created.

Important:

- Open **Visual Studio Code (VS Code)**.
- Navigate to the **server** directory.
- Open the **Integrated Terminal**.

Step 2: Initialize the Node.js Project

Run the following command to initialize the backend project.

```
npm init -y
```

Important:

- Creates the **package.json** file.
- Initializes the Node.js project with default settings.
- Prepares the project for installing dependencies.

Step 3: Create the Main Server File

Create the main server file named:

server.js

This file acts as the **entry point** of the backend application. It initializes the Express server, configures middleware, connects to MongoDB, and starts the server.

Important:

- Create **server.js** inside the **server** folder.
- This file is the **entry point** of the backend application.

Step 4: Create the Project Folder Structure

Create the following folders inside the **server** directory.

```
server
|
├── controllers
├── models
├── routes
├── config
├── middlewares
└── uploads
```

Folder Description

- **controllers** – Contains the business logic for handling requests.
- **models** – Stores MongoDB schemas and database models.
- **routes** – Defines API endpoints for the application.
- **config** – Stores database connection and configuration files.
- **middlewares** – Contains authentication and custom middleware functions.
- **uploads** – Stores uploaded property images.

Important:

- Organize the backend using a **modular folder structure**.
- Separate **routes**, **controllers**, and **models** for better maintainability.

Step 5: Install Required Dependencies

Install the required backend packages.

```
npm install express mongoose dotenv cors bcryptjs jsonwebtoken multer nodemailer
```

Install the development dependency.

```
npm install --save-dev nodemon
```

Important:

- **Express** – Web application framework.
- **Mongoose** – MongoDB Object Data Modeling (ODM).
- **dotenv** – Loads environment variables.
- **cors** – Enables cross-origin resource sharing.
- **bcryptjs** – Encrypts user passwords.
- **jsonwebtoken** – Implements JWT authentication.
- **multer** – Handles file uploads.
- **nodemailer** – Sends emails.
- **nodemon** – Automatically restarts the server during development.

Step 6: Start the Backend Server

Run the following command to start the backend server.

```
npm run dev
```

or

```
node server.js
```

Important:

- Starts the **Express.js** server.
- Connects to the **MongoDB** database.
- Makes the backend APIs available for client requests.

Step 7: Verify the Server

If the server starts successfully, the terminal displays a message similar to:
Server running on Port 5000

MongoDB Connected Successfully Open the following URL in your browser or Postman:
<http://localhost:5000>

The backend server is now ready to communicate with the frontend application.

Backend Setup Workflow



Backend/Server Setup Workflow

